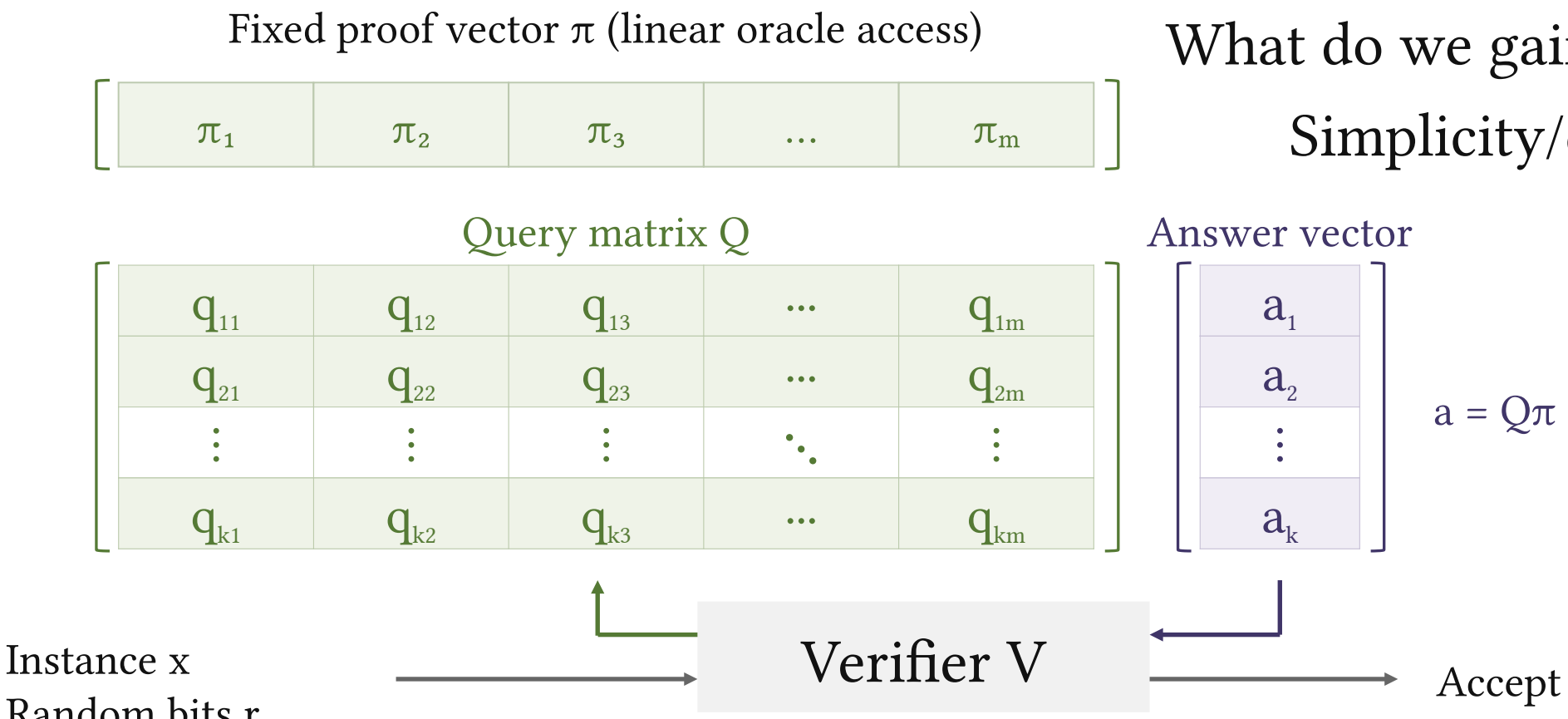
Dot Product Proofs of Powers and Applications

Nir Bitansky, **Saroja Erabelli**, Yuval Ishai, Ron D. Rothblum, William Wang

NYU NYU AWS, Technion Succinct NYU

Linear PCPs [IKO07, BCIOP13]

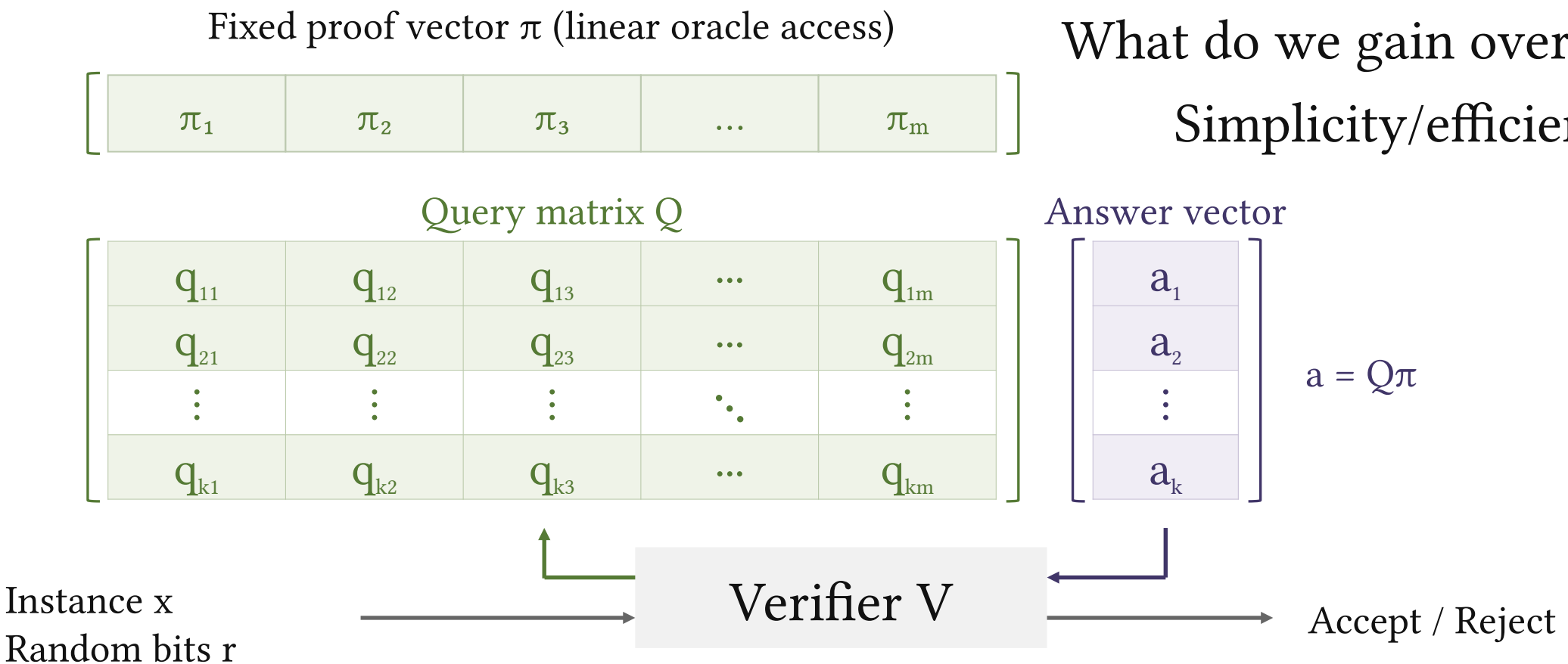


Each query returns a linear combination

$a = Q\pi, \quad a_i = \langle q_i, \pi \rangle$

Rows of Q are query vectors over a finite field.

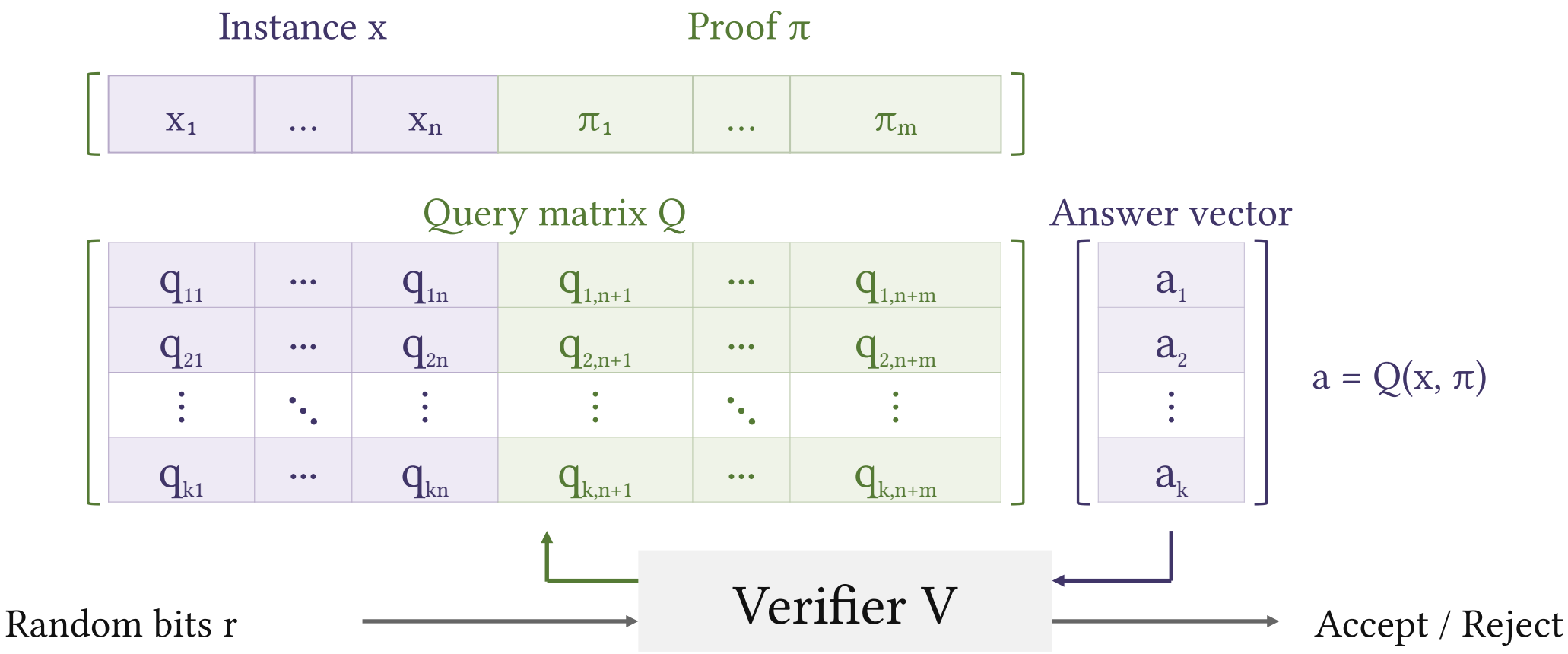
Linear PCPs [IKO07, BCIOP13]



Central tool in group-based succinct arguments [GGPR13, BCIOP13, Groth16]

Very small proofs: **3 queries → 3 group elements**

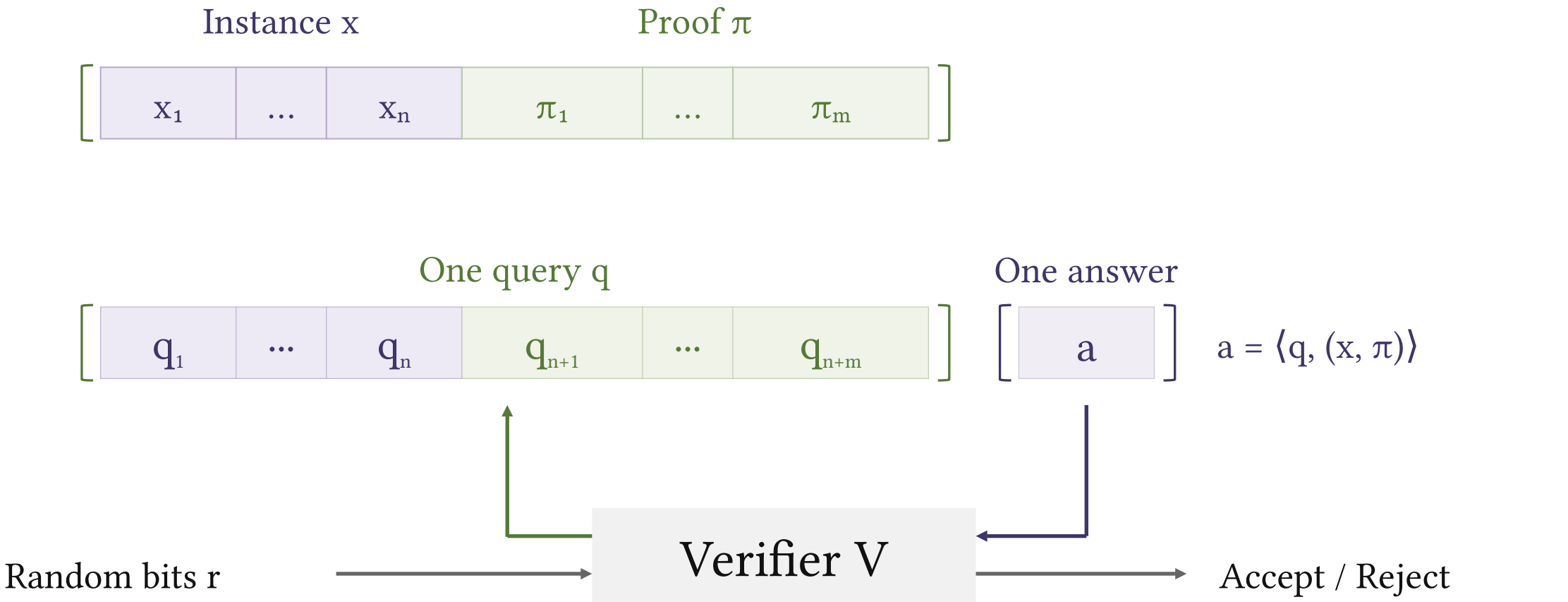
Fully Linear PCP (FLPCP) [BBCGI19]



Verifier only get linear query access to x

$$a = Q(x, \pi), \quad a_i = \langle q_i, (x, \pi) \rangle$$

Dot Product Proof (DPP) [BCIOP13, BIOW20, BHIRW24]



Verifier only gets one linear query

$a = \langle q, (x, \pi) \rangle$

→ 1 group element arguments

Why study DPPs?

Succinct arguments

multi-query FLPCPs	multi-group element argument
DPPs	**one group element argument**

Hardness of approximation

PCPs	hardness of approximation for problems with **locality**
DPPs	hardness of approximation for problems with **linear equations**

Why study DPPs?

Hardness of approximation

Hardness-preserving reductions (used to call this ETH-hardness):

- DPP gives Gap-MAXLIN is as hard as SAT with no loss [BHIRW24]

- DPP gives Gap-CVP with constant factor approximation is as hard as SAT with no loss [AGMZ26]

- OpenAI hardness result for Gap-CVP with $n^{1/400}$-factor approximation used DPP-related technology
  - Reed-Solomon power sum constraints
  - Algebraic reconstruction

No complicated PCP machinery

What are the gaps in existing DPPs? [BHIRW24]

Small-field DPP

Hardness of approximation

Issue: Proof size grows with the field

Large-field DPP

Laconic arguments and SNARGs

Issues:

- Complicated packing
- Not field-agnostic: encodes one field into another field

Both of these approaches inherently rely on the field

This work: Field-Agnostic DPP

Native DPP for Arithmetic Circuit SAT

- Clean, direct construction
- Essentially field-agnostic
- Zero-knowledge for free
- Proof size: independent of $|F|$, depends on $\text{poly}(1/\varepsilon)$

Applications

Laconic arguments/SNARGs

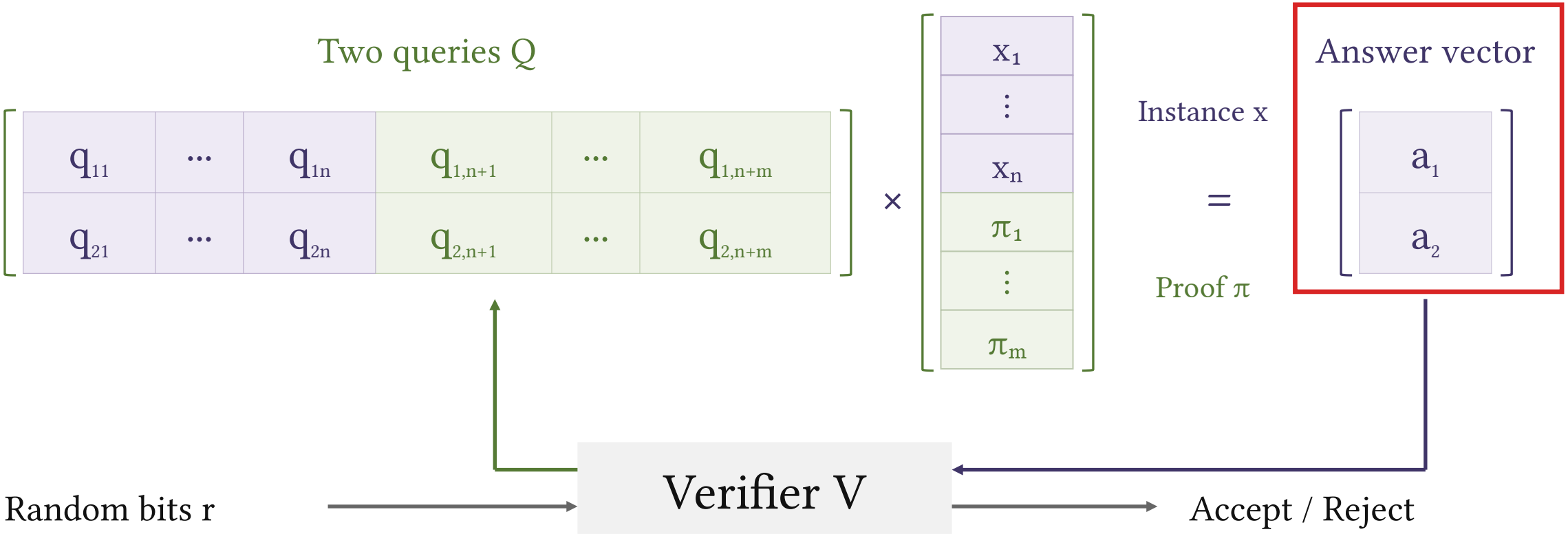
concrete improvements

Hardness of approximation

for Generalized MAXLIN

Starting Point: Two-Query FLPCP for AC-SAT

[BHIRW24]

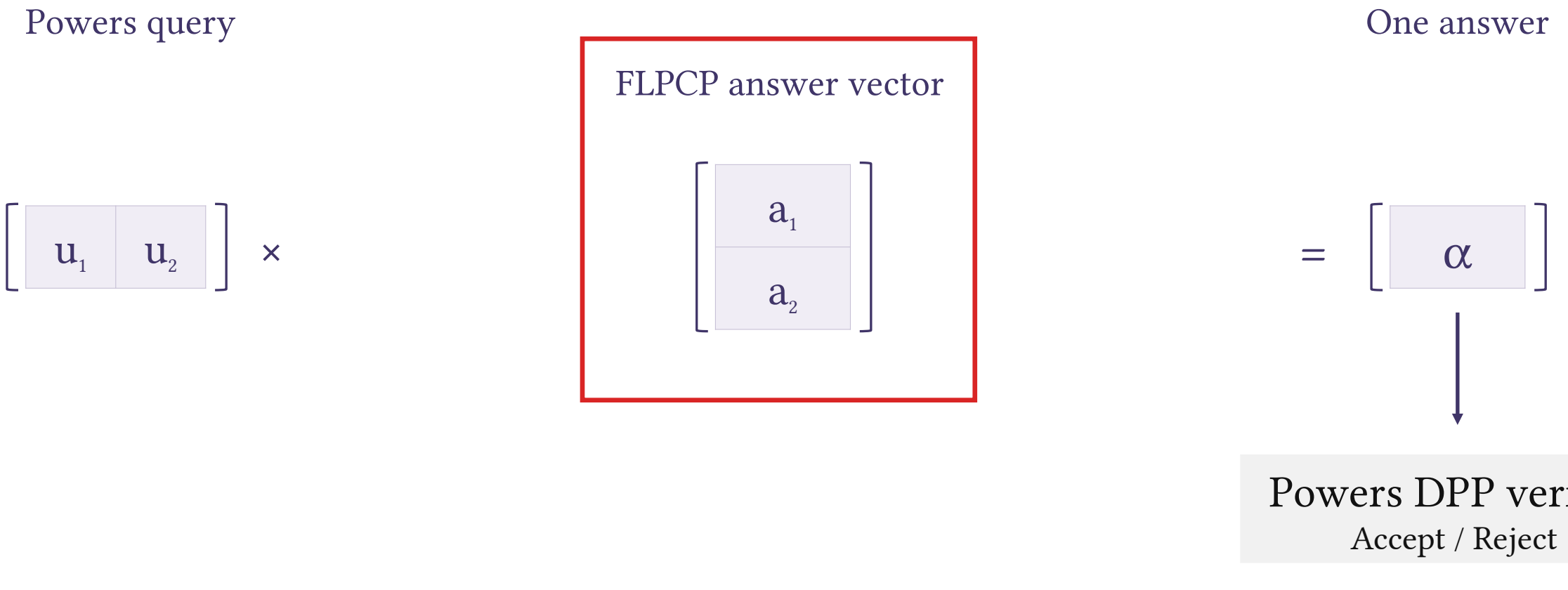


Powers accepting set: **Verifier accepts answers of the form (a, a^2)**

Can check with a single exponentiation

What if we had a DPP for Powers?

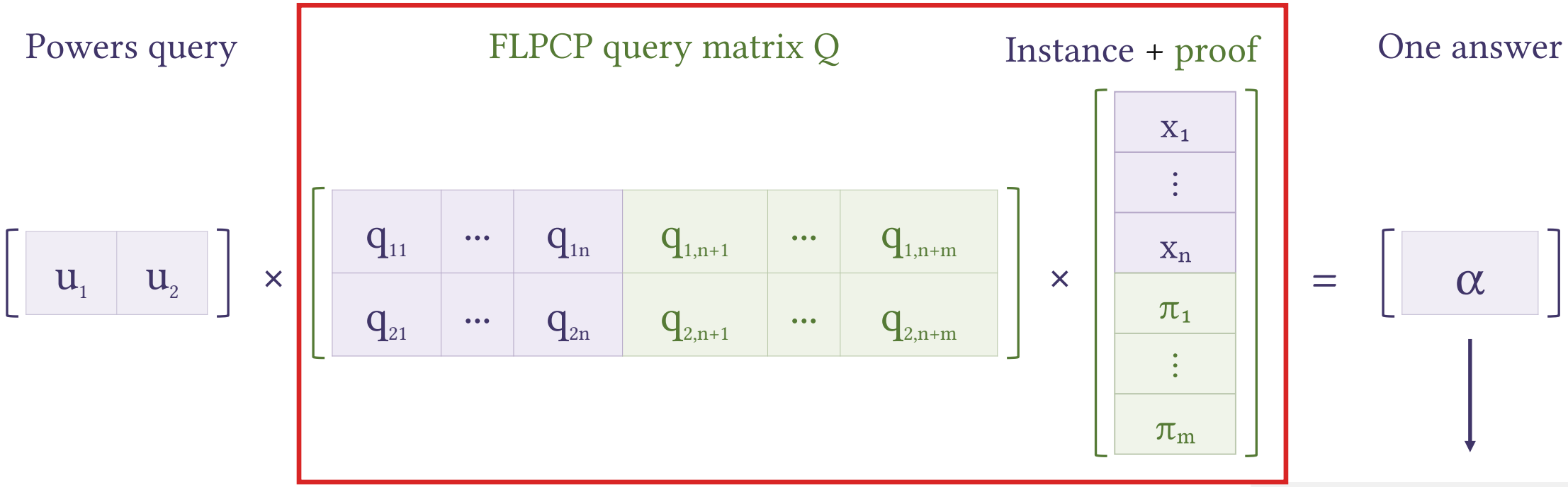
Apply the Powers DPP to the FLPCP answer vector.



Note: This illustration requires the Powers DPP to be **proofless**

DPP for Powers ⟹ DPP for AC-SAT [BHIRW24]

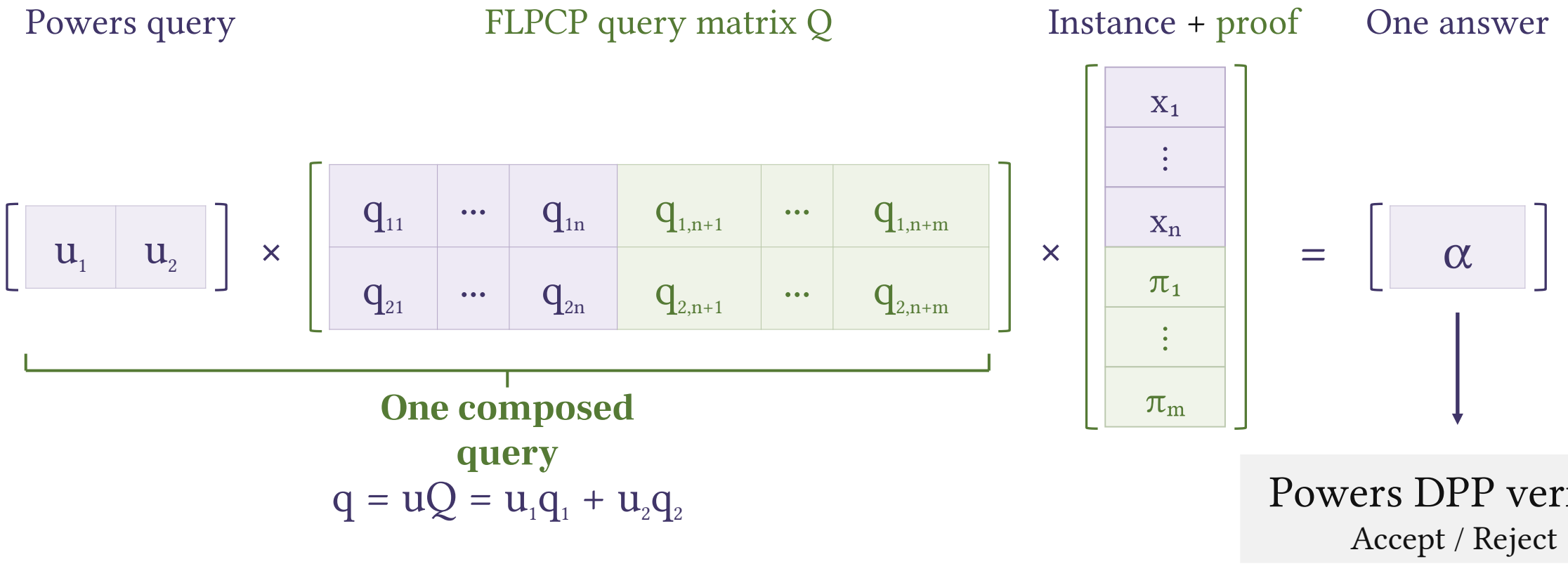
Compose the Powers query with the FLPCP query matrix.



Note: This illustration requires the Powers DPP to be **proofless**

DPP for Powers ⟹ DPP for AC-SAT [BHIRW24]

Compose the Powers query with the FLPCP query matrix.



Note: This illustration requires the Powers DPP to be **proofless**

Powers DPP

The language

$$\mathrm{Pow}_d(F) = \{ (1, a, a^2, \ldots, a^d) : a \in F \}$$

$$R_d = \{ u^d : u \in F \} \text{ — the dth powers}$$

Our goal

$$(1, a, a^2, \ldots, a^d) \longrightarrow (r + a)^d$$

Always a dth power

Construction shown over $F = F_p$: p prime, d prime, and $d \mid (p - 1)$.

Powers DPP

1 Choose a random r ∈ F

$$q(r) = \left(r^d, \binom{d}{1} r^{d-1}, \quad \binom{d}{2} r^{d-2}, \ldots, 1\right)$$

2 Get query answer $\alpha = \langle q(r), x \rangle$

3 Test for a dth power $\alpha \in R_d$

Completeness

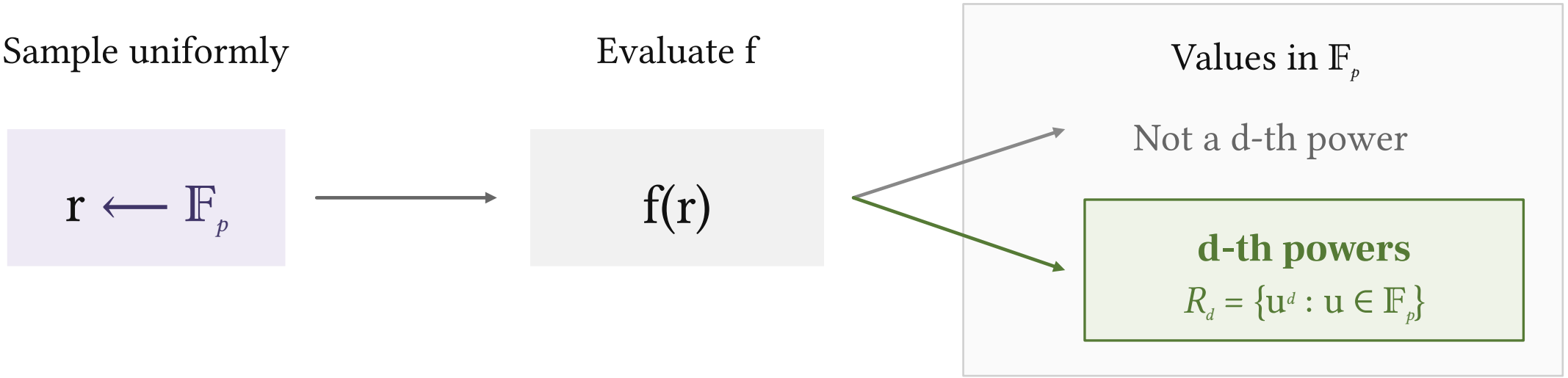
$$\langle q(r), (1, a, a^2, \ldots, a^d) \rangle = (r + a)^d$$

Powers DPP Soundness

Reduces to a clean number theoretic question:

What is the probability f(r) is a dth power over $\mathbb{F}_p$?

assuming $f(r) \neq g(r)^d$



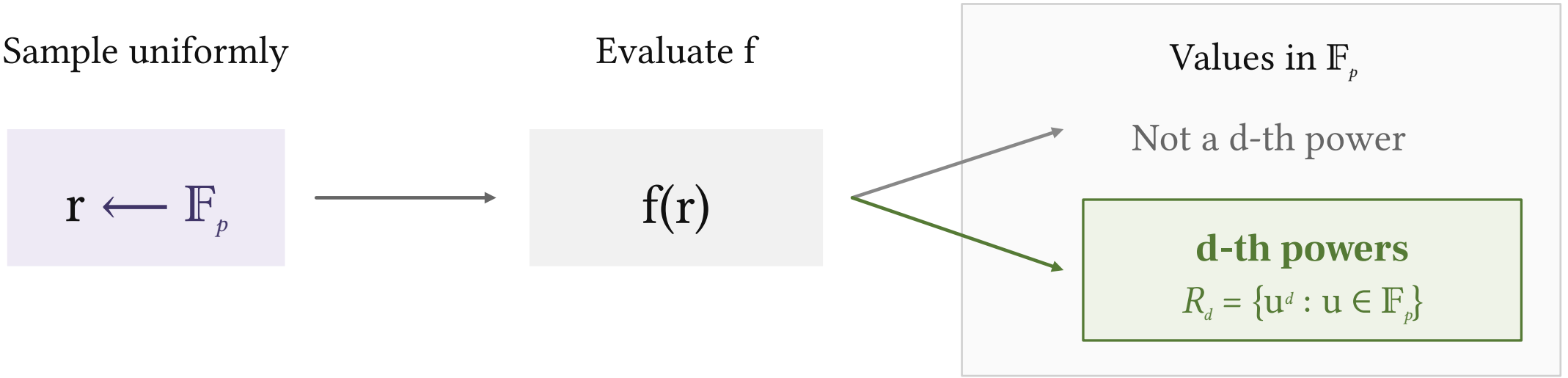
$$Pr_{r \leftarrow \mathbb{F}_p}[f(r) \in R_d] = ?$$

Powers DPP Soundness

Reduces to a clean number theoretic question:

What is the probability f(r) is a dth power over $\mathbb{F}_p$?

assuming $f(r) \neq g(r)^d$



$$Pr_{r \leftarrow \mathbb{F}_p}[\, f(r) \in R_d \,] \approx 1/d$$

Field-Agnostic DPP for Arithmetic Circuits

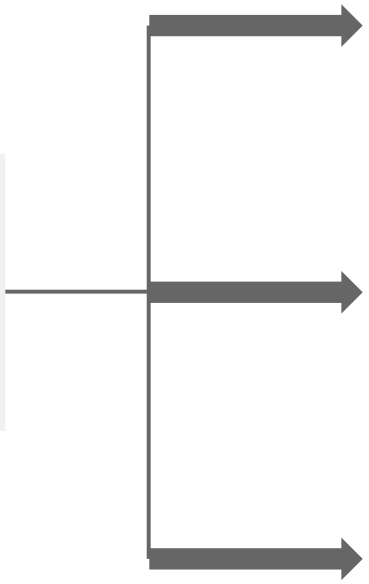
Theorem.

A DPP for AC-SAT over $\mathbb{F}_p$ for circuits with S multiplication gates for all prime d such that d | p - 1

Proof length	**Soundness error**
$O(d^2 S)$	$1/d + o(1)$

Applications

DPP for Powers



More efficient one-group element laconic arguments with preprocessing

More efficient single-ciphertext designated-verifier reusable SNARGs

Hardness of approximation for Generalized MAXLIN

Applications

DPP for Powers

There's a better, much simpler solution by ChatGPT based on FLPCPs

~~More efficient one-group element laconic arguments with preprocessing~~

More efficient single-ciphertext designated-verifier reusable SNARGs

Hardness of approximation for Generalized MAXLIN

Applications

DPP for Powers

~~More efficient one-group element laconic arguments with preprocessing~~

More efficient single-ciphertext designated-verifier reusable SNARGs

Hardness of approximation for Generalized MAXLIN

Standard MAXLIN

Input: m linear equations $\langle a_i, z\rangle + b_i = 0$ over $\mathbb{F}_p$.

Goal: choose $z \in \mathbb{F}_p^n$ to maximize the fraction satisfied.

Example over $\mathbb{F}_2$

$$z_1 + 1 = 0$$
$$z_2 = 0$$
$$z_1 + z_2 = 0$$

At $z = (1,0)$, the outputs are

0	satisfied
0	satisfied
1	not satisfied

This assignment satisfies 2 of the 3 equations.

Objective: $\mathrm{OPT} = \max_{z \in \mathbb{F}_p^n} \frac{|\{i : \langle a_i, z\rangle + b_i = 0\}|}{m}$

Generalized MAXLIN [Bhangale-Zhang26]

Choose $z \in \mathbb{F}_p^n$ to satisfy as many constraints as possible.

Standard MAXLIN

$\langle a_i, z \rangle + b_i = 0$

→

Generalized MAXLIN(R)

$\langle a_i, z \rangle + b_i \in R$

Powers CSP: $R = R_d$ $^d : u \in \mathbb{F}$

Gap-MAXLIN$_{s,c}(R)$
$0 \le s < c \le 1$



Reduction: AC-SAT to MAXLIN(R_d)

AC-SAT

size n over $\mathbb{F}_p$

DPP gives poly-time reduction



$N = O(n)$ variables

Gap-MAXLIN(R_d)

N variables $\mathbf{z} \in \mathbb{F}_p^N$

satisfiable → **YES:** OPT ≥ 1

unsatisfiable → **NO:** OPT ≤ $1/d + \varepsilon$

Hardness-preserving

AC-SAT: no $p^{o(n)}$ algorithm → gap-MAXLIN(R_d): no $p^{o(N)}$ algorithm

AC-SAT: no $p^{o(n/\log(n))}$ algorithm → gap-MAXLIN(R_d): no $p^{o(N/\log(N))}$ algorithm

Open Questions

1) More applications/implications of DPPs

2) Dream DPP parameters
   - Currently proof size scales with $1/\varepsilon^2$
   - Can we achieve proof size scaling with polylog$(1/\varepsilon)$?

Thank you!